

Software Manual

Table of Contents

Current Page:

- [Introduction](#)
 - [Project Goal](#)
 - [Project Scope](#)
- [Terminology](#)
- [Get Started](#)
 - [Build Instructions](#)
 - [Coding Conventions](#)
 - [Git Conventions](#)

Separate Pages:

- [User Authentication](#)
- [Projects](#)
- [Data Display](#)
- [Database](#)
- [Templates](#)

Further Documentation:

- [Further Documentation](#)
-

Introduction

Welcome to the **Collaborative Budget Management Software Manual**. This is a guide for developers working on the system, providing an overview of the architecture, core features and best practices for development and maintainability.

Project Goal

The goal of the software is to provide an efficient and scalable budget management system which allows users to:

- Create and manage budgets for projects.
- Invite project collaborators to projects to enter/view budget data.
- View budget breakdowns for projects.
- Allow for projects to be dynamically created based on the project template they are using.

Project Scope

The system is a web-based application built using, **React, TypeScript, Supabase, Shadcn and Clerk** supporting:

- Multi-user access with role-based permissions.
- Dynamic project creation using custom-made JSON templates.
- A flexible database schema that supports any properly structured template.

Terminology

Within this software manual, specific terminology to the project may be used frequently. This section defines these key terms to ensure clarity and consistency for all developers working on the system.

- **Project Template** - A JSON-based structure defining the categories, fields, calculations.
- **Principal Investigator (PI)** - The lead responsible for overseeing the project. By default, this is the project creator.
- **Institution Lead** - A leader within an institution, managing project-related activities and users from their institution.
- **Institution Collaborator** - A user working on the project under a specific institution.

Get Started

Build Instructions

To start developing, following these steps to set up the project environment and build the application locally:

1. Clone the Repository

```
git clone https://projects.cs.nott.ac.uk/comp2002/2024-2025/team45_project.git
cd team45_project
```

2. Install Dependencies

Ensure that you have [Node.js](#) and npm installed. Then run, this command to install the required packages:

```
npm install
```

3. Run the Development Server

Start the development server by running:

```
npm run dev
```

This will launch the application at <http://localhost:5173/>.

4. Run Tests

To run the unit tests and ensure everything is working correctly, run the command:

```
npm run test
```

Coding Conventions

Coding conventions that must be followed when developing this project can be found [here](#).

Git Conventions

Git conventions that must be followed when developing this project can be found [here](#).

Further Documentation

To gain a deeper understanding of different sections of the project, read more of the software manual [here](#).

Any further documentation including: prototypes, requirements and UML diagrams can be found [here](#).

[↑ Back to top](#)

User Authentication

Table of Contents

- [User Authentication](#)
 - [Authentication Flow](#)
 - [User Signup & Login](#)
 - [Implementation](#)
 - [Clerk Authentication Setup](#)
 - [Handling User Login and Database Storage](#)
 - [Clerk Authentication Features](#)
 - [Email Verification Code Login](#)
 - [Session Management](#)
 - [Signed In & Signed Out Components](#)
 - [User Authorization & Role Management](#)
 - [Protected Routes](#)
 - [User Session Handling](#)
 - [Logout Handling](#)
 - [Adavanced Database Setup with Supabase](#)
 - [Example Schema](#)
 - [Troubleshooting](#)
 - [Summary](#)

Introduction

This manual provides an in-depth guide on how authentication is handled in the application using **Clerk** for login/signup and **Supabase** for user storage. Clerk manages user authentication, while Supabase serves as the

database backend to store user information.

The app uses **email verification codes** for login, ensuring secure access. Clerk handles authentication seamlessly, while Supabase maintains user records.

Authentication Flow

User Signup & Login

1. The user visits the application.
2. If the user is signed out, they are redirected to the **Login Page**.
3. Clerk manages the **email verification** process by sending a **one-time code** to the user's email.
4. Once verified, Clerk retrieves the **authenticated user data**.
5. The app checks if the user exists in the **Supabase database**.
6. If the user does not exist, a new record is **created** in Supabase.

Implementation Details

Clerk Authentication Setup

To integrate Clerk, the application includes the **ClerkProvider** component, which wraps the main app component:

```
import { ClerkProvider } from "@clerk/clerk-react";

const App = () => (
  <ClerkProvider publishableKey="your-clerk-publishable-key">
    <YourAppComponents />
  </ClerkProvider>
);

export default App;
```

The App.tsx file handles user authentication using Clerk:

```
type Invitation = {
  invitedID: number;
  inviteID: number;
  projectID: number;
  roleID: number;
  status?: string;
  email?: string;
};

function App() {
  // Get currently logged in user from Clerk
  const { user } = useUser();
  const [userID, setUserID] = useState<number | null>(null);
  const [invitations] = useState<Invitation[]>([]);
```

```
// Hook to handle user login once user state is set from Clerk
useEffect(() => {
  if (user) {
    const fetchUserID = async () => {
      const storedUserID = await HandleUserLogin(user);
      setUserID(storedUserID);
    };
    fetchUserID();
  }
}, [user]);

const userEmail = user?.primaryEmailAddress?.emailAddress || "";

return (
  <Router>
    {/* Signed out View */}
    <SignedOut>
      <Login />
    </SignedOut>
    {/* Signed in View */}
    <SignedIn>

```

Handling User Login and Database Storage

The `HandleUserLogin` function ensures that once a user logs in via Clerk, they are stored in Supabase:

```
const HandleUserLogin = async (user: any) => {
  console.log("HandleUserLogin was called!", user);
  if (!user) {
    console.error("No user found in Clerk.");
    return;
  }

  const email = user.primaryEmailAddress?.emailAddress;
  const username = user.username || user.firstName || email;
  const firstName = user.firstName || "Unknown";
  const lastName = user.lastName || "Unknown";

  // If missing email or username
  if (!email || !username) {
    console.error("Required user information missing.");
    return;
  }

  let userId;

  try {
    const { data, error } = await supabase
      .from("Users")

```

```

        .select("userID")
        .eq("email", email);

    if (error) {
        throw new Error(`Error checking user existence: ${error.message}`);
    }

    // If user in db exists
    if (data && data.length > 0) {
        userId = data[0].userID;
        console.log(`User found: ${userId}`);
    }
    // If no user found in db
    else {
        // Insert new user info into db
        const { data: newUser, error: insertError } = await supabase
            .from("Users")
            .insert({
                firstName,
                lastName,
                email,
                username,
            })
            .select("userID")
            .single();

        if (insertError) {
            throw new Error(`Error creating user: ${insertError.message}`);
        }

        userId = newUser?.userID;
        console.log(`New user created: ${userId}`);
    }

    localStorage.setItem("userID", userId);
    console.log(`User session stored with ID: ${userId}`);
    return userId;
} catch (err) {
    console.error(
        `Error handling user login (userID ${userId}): ${err.message}`
    );
}
};

```

This ensures that every new user is recorded in Supabase.

Clerk Authentication Features

Email Verification Code Login

1. Clerk automatically sends a one-time code to the user's email.
2. The user enters the code to authenticate and gain access.

Session Management

1. Clerk automatically manages user sessions.
2. Users remain logged in unless they manually log out or the session expires

Signed In & Signed Out Components

Clerk provides simple conditional rendering for authentication-based UI elements:

```
import { SignedIn, SignedOut } from '@clerk/clerk-react';

<SignedIn>
  <p>Welcome, you are signed in!</p>
</SignedIn>

<SignedOut>
  <p>Please sign in to continue.</p>
</SignedOut>
```

User Authorization & Role Management

Protected Routes

Certain routes require authentication. We wrap protected pages with the SignedIn component:

```
<SignedIn>
  <Route path="/dashboard" element={<Dashboard />} />
</SignedIn>
```

If a user is not signed in, they are automatically redirected to the Login Page.

User Session Handling

Clerk provides an active session object, which can be used as follows:

```
import { useAuth } from '@clerk/clerk-react';

const { sessionId, userId } = useAuth();
console.log(`Current session ID: ${sessionId}, User ID: ${userId}`);
```

1. The app retrieves the user session from Clerk.
2. The session is stored in localStorage for convenience.

```
localStorage.setItem("userId", userId);
console.log(`User session stored with ID: ${userId}`);
```

```
return userId;
```

```
const storedUserId = localStorage.getItem("userID");
```

Logout Handling

Clerk provides a built-in profile dropdown menu with a sign-out option:

```
import { UserButton } from "@clerk/clerk-react";

<UserButton />;
```

By default, a small profile icon appears in the top left of the app, allowing users to log out.

Advanced Database Setup with Supabase

The database schema includes tables for users, roles, projects, and institutions. Key relationships:

1. Users are stored in Supabase with a unique ID.
2. Institutions store organizational details.
3. Projects are linked to institutions and users.
4. UserRoles define access levels.

Example Schema:

```
CREATE TABLE Users (
  userID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  firstName VARCHAR,
  lastName VARCHAR,
  email VARCHAR UNIQUE NOT NULL,
  username VARCHAR
);

CREATE TABLE Projects (
  projectID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  principalInvestigatorID BIGINT REFERENCES Users(userID),
  projectName TEXT NOT NULL,
  projectAcronym TEXT
);

CREATE TABLE UserInstitutionProject (
  userInstitutionProjectID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  userID BIGINT REFERENCES Users(userID),
  institutionID BIGINT,
  roleID BIGINT,
  projectID BIGINT REFERENCES Projects(projectID)
```



```
);
```

This allows advanced permissions and access control.

Troubleshooting

Issue: User Not Found in Database

Ensure the Supabase "Users" table exists. Check that the email verification process is complete.

Issue: User Can't Login

Verify the user’s email verification status in the Clerk dashboard. Ensure the user's email exists in Clerk and Supabase.

Summary

Clerk handles authentication (login/signup). Supabase stores user records and manages roles. Email verification codes are used for login. Protected routes restrict access based on authentication and roles. Session management ensures users remain logged in.

This manual provides everything needed to implement a secure authentication and user management system using Clerk and Supabase.

[↑ Back to top](#)

Projects

Table of Contents

- [Projects](#)
 - [Project Creation](#)
 - [Project Display](#)
 - [Project View & Data Entry](#)
 - [Interfaces Used](#)
 - [Data Entry Flow](#)
 - [Managing Collaborators](#)
 - [Components Used](#)
 - [Invitations](#)
 - [Invitation Flow](#)
 - [Managing Invites](#)
 - [Summary](#)

The Projects section is a fundamental component of the system, enabling users to create, manage, and collaborate on projects within their institution. Users can define project metadata, assign roles, and send invitations to other collaborators. This section communicates with the backend database through Supabase’s JavaScript client for all project-related CRUD operations.

Project Creation

Project creation is initiated from the `CreateProject.tsx` page. When a user creates a project, the following steps are performed:

1. Institution Selection

Users select an institution from their associated list retrieved from the database using the `getUsersInstitutions()` function.

2. Metadata Entry

Users input:

- `projectName`: A user-defined name for the project.
- `projectAcronym`: A short acronym for quick reference.

3. Project Creation

The system calls `createProjectQuery()` with the selected institution ID and project details. A new row is inserted into the `Project` table.

If successful, the newly created `projectID` is stored in `localStorage` for use across other pages.

4. Category Setup

The selected template is parsed to extract categories. These are inserted into the database via the `checkAndAddCategories()` function to ensure the correct structure for future data entry.

This entire flow is triggered when the user clicks the **Create Project** button.

Project Display

On the `Projects.tsx` page, the system retrieves all projects linked to the user and displays them in a card-based UI. Each card includes:

- Project name
- Acronym
- Navigation button to the full project view

Routing is handled using React Router and links to the `/project-view` route.

Project View and Data Entry

The `ProjectView.tsx` page allows users to view and interact with a project's budget and category entries. Depending on permissions, users may be in read-only mode or full-edit mode.

Interfaces Used

- **DynamicBudgetForm.tsx**
Provides a tab-based UI to navigate through each budget category.
- **GenericForm.tsx**
Dynamically generates input forms based on field definitions from the template file.

Data Entry Flow

1. The user selects a category tab.
2. All editable fields are rendered.
3. On submission:
 - Data is validated.
 - The `createEntry()` function is called to insert entries into the database.

Feedback is given to the user upon successful or failed submission.

Managing Collaborators

Project collaboration is an integral feature of the system. Users can add or remove collaborators and assign them specific roles within a project.

Components Used

- **InviteUser.tsx**: Allows the project owner to send invitations to new members.
- **ManageCollaborators.tsx**: Displays and manages current collaborators and their roles.
- **CollaboratorList.tsx**: Shows a read-only list of collaborators, their emails, and assigned roles.

Collaborator data is fetched by querying the `UserInstitutionProject`, `Users`, and `Roles` tables using Supabase.

Sending and Accepting Invitations

Invitations are handled through the `Invites` table in the database. This enables tracking of pending, accepted, and rejected invites.

Invitation Flow

1. The user enters the invitee's email and selects a role.
2. A new row is added to the `Invites` table with:
 - `invitedID`
 - `projectID`
 - `roleID`
 - `status = "pending"`

Managing Invites

- **Invitations.tsx** displays all pending and sent invitations.
- Users can:

- Accept an invite: This creates a new record in the `UserInstitutionProject` table and updates the invite status to `"accepted"`.
- Reject an invite: This updates the status to `"rejected"`.

The accepting user must also select an institution to associate with the project upon acceptance.

Access Control and Roles

Each user in a project is associated with a role stored in the `Roles` table. When fetching collaborator info or rendering the UI, the system uses the user's role to control visibility of editing tools and settings.

Summary of Key Functions and Files

Feature	File/Function
Create Project	<code>CreateProject.tsx</code> , <code>createProjectQuery()</code>
Fetch Institutions	<code>getUsersInstitutions()</code>
Add Categories	<code>checkAndAddCategories()</code>
Project View UI	<code>ProjectView.tsx</code>
Dynamic Form	<code>DynamicBudgetForm.tsx</code> , <code>GenericForm.tsx</code>
Data Entry	<code>createEntry()</code>
Invite Users	<code>InviteUser.tsx</code> , <code>Invitations.tsx</code>
Manage Roles	<code>ManageCollaborators.tsx</code>
Show Collaborators	<code>CollaboratorList.tsx</code>

This Projects section handles everything from inception to collaboration, serving as the operational core of the system.

[↑ Back to top](#)

Data Display

Table of Contents

- [Data Display](#)
 - [Data Retrieval](#)
 - [Data Formatting](#)
 - [Table Display](#)

Data display works in three parts:

1. Retrieve the data from the database

2. Format the data
3. Display the data

Data Retrieval

Data retrieval works via a chain of fetch requests to the database, starting from the Category ID for the category we are fetching data for and ending with a 2D array of (unordered) data. These functions are all combined into one master function within the `"queryFunctions.tsx"` file, located at the bottom.

```
export async function getCategoryDataCatOnly(
  categoryID: number
): Promise<string[][]> {
  const fieldIDs = await getFieldIDs(categoryID);
  const catHeaders = await getCategoryHeaders(categoryID);
  const fieldData = await getFieldData(fieldIDs); // Check if fieldData is empty
  or its first element is undefined.

  if (!fieldData || fieldData.length === 0 || !fieldData[0]) {
    return [catHeaders]; // Return only the headers if there's no data.
  }

  const result = formatData(catHeaders, fieldData);
  return result;
}
```

The overall process is the following:

1. Using the Category ID, query the database to find every Field ID linked to said Category.
2. Query the database to get the heading/title of every Field, and store in a separate array called `catHeaders`.
3. Then query every Field ID, creating a 2D array that stores the results of each query in separate arrays. Store within `fieldData`.
4. If `fieldData` is empty, meaning there is no data added to this category yet, only return the headings of each category (`catHeaders`).
5. Otherwise, format and combine all data (as it is currently unordered), and return the result.

All of these steps typically make use of an asynchronous function that queries the database, and handles the data in some way. See the following:

```
async function getCategoryHeaders(categoryID: number): Promise<string[]> {
  const { data, error } = await supabase
    .from("CategoryFields")
    .select("fieldName")
    .eq("categoryID", categoryID);
  if (error) {
    console.error("Unable to fetch headings for CategoryID:", categoryID);
    return [];
  }
}
```

```
    return data.map((item) => cleanString(item.fieldName));
  }
```

Here is the function for fetching all the category headers. You can clearly see that it first queries the database, as well as checking for errors, before mapping the results to an array. This is the typical format all of these functions follow, with all functions within the `"queryFunctions.tsx"` file formatted with JSDoc:

```
/**
 * Fetches all FieldNames for the Field of a given Category.
 * @param {number} categoryID ID of category to fetch FieldNames for.
 * @returns {Promise<string[]>} A promise that resolves to an array of FieldNames.
 *
 * @example
 * const categoryHeaders = await getCategoryHeaders(categoryID);
 */
```

This is the JSDoc for the above function.

Data Formatting

When retrieving data for each field, the code will fetch the data for one field at a time. This is great for ease of use and speed, but leaves the overall data unordered. Take the following example:

```
[
  ["name1", "name2", "name3", "name4", "name5", "name6"],
  [age1, age2, age3, age4, age5, age6],
  [gender1, gender2, gender3, gender4, gender5, gender6],
  [salary1, salary2, salary3, salary4, salary5, salary6],
];
```

In the above example, the fieldData we truly want is top-down. That is, we want `["name1", age1, gender1, salary1]` to be the first array, not all the names, then all the ages, genders, and so on.

To do this, we use the `formatData` function:

```
function formatData(
  categoryHeaders: string[],
  fieldData: string[][]
): string[][] {
  const transposed = fieldData[0].map((_, colIndex) =>
    fieldData.map((row) => row[colIndex])
  );
  transposed.unshift(categoryHeaders);

  return transposed;
}
```

This function does 2 things:

1. Flip the rows and columns of `fieldData`, storing the result in a new 2D array called `transposed`.
2. Push the `catHeaders` array to the front of `transposed`

In the example previously mentioned, with a `catHeaders` of `["Name", "Age", "Gender", "Salary"]`, we would end up with the following:

```
[
  ["Name", "Age", "Gender", "Salary"],
  ["name1", age1, gender1, salary1],
  ["name2", age2, gender2, salary2],
  ["name3", age3, gender3, salary3],
  ["name4", age4, gender4, salary4],
  ["name5", age5, gender5, salary5],
  ["name6", age6, gender6, salary6],
];
```

You can see now that the data is ordered, and already is beginning to look like a table. All that is left is to display it to the user.

Table Display

The final step now is to display the data back to the user, in a way that can account for any size of 2D array. This is handled within a modular component, that can be used where required. This component is contained within the `"categoryDisplay.tsx"` file, which is documented with JSDoc.

The main section, `lines 26-61`, handles the logic for this:

```
const CategoryDisplay: React.FC<{ data: string[][] | null | undefined }> = ({ data
}): JSX.Element => {
  // Handle invalid data
  const headers = useMemo(() => (data && data.length > 0 ? data[0] || [] : []),
[data])
  const tableItems = useMemo(() => (data && data.length > 1 ? data.slice(1) : []),
[data])

  const columns = useMemo<ColumnDef<Record<string, string>>[]>(() => {
    return headers.map((header, index) => ({
      accessorKey: header.toString() || `column${index}`,
      header: () => <div>{header}</div>,
      cell: ({ row }) => <div>{row.getValue(header.toString() ||
`column${index}`)}</div>,
    })))
  }, [headers])

  // Transform data into an array of objects
  const tableData = useMemo(() => {
    return tableItems.map((row) => {
```

```
const rowData: Record<string, string> = {}
headers.forEach((header, index) => {
  rowData[header.toString() || `column${index}`] = row[index] || ""
})
return rowData
})
}, [headers, tableItems])

// Create table instance with pagination
const table = useReactTable({
  data: tableData,
  columns,
  getCoreRowModel: getCoreRowModel(),
  getPaginationRowModel: getPaginationRowModel(),
  initialState: {
    pagination: {
      pageSize: 10,
    },
  },
},
})
```

First, the 2D array is sliced along the first item, separating the array of headings, `catHeaders`, and the 2D array of data, `fieldData`.

Then, the data is memoized, using the `useMemo` react hook, to store `catHeaders` into `columns`, and `fieldData` into `tableItems`. Each individual row is stored within a `rowData` object, which combined together make up `tableData`, which is the overall data for the table.

This is then used to create a table instance, using pagination, with `tableData` as our base. The rest of the "`categoryDisplay.tsx`" file is formatting, using `shadcn` components for design.

[↑ Back to top](#)

Database

Table of Contents

- [Database](#)
 - [ERD Diagram](#)
 - [Connection](#)
 - [Project API Keys](#)
 - [Schema Overview](#)
 - [Data Entry](#)
 - [Steps for Data Entry](#)
 - [Database Queries](#)

The database is the core data storage component of the system, managing both user, institution and project data in their respective tables. It uses **PostgreSQL**, hosted on **Supabase**.

ERD Diagram

An up-to-date copy of the ERD diagram for the database currently being hosted on Supabase can be found [here](#).

Connection

The connection to the database is established once the website is launched, this can be found in [database.tsx](#). The system makes use of **Supabase's JavaScript Client** to allow the code to interact with the database via REST APIs.

As seen in [database.tsx](#), connection to the database is made using the **Supabase Client**.

```
import { createClient } from "@supabase/supabase-js";

const supabaseUrl = "https://xybccoipttcvmdniwysj.supabase.co";
const supabaseKey =

"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Inh5YmNjb2lw dHRjdm1kbml3eXNqIiwicm9sZSI6ImFub24iLCJpYXQiOiJlYXN0IiwiaWF0Ij0iMTY5ODc2NH0.qft8IvKBxpEzW7Uh1D4uDdGafhHzbh7fWlfi17B5nKA";

const supabase = createClient(supabaseUrl, supabaseKey);
```

Project API Keys

As of **02/04/2025**, the following API keys are utilised within the application to perform CRUD operations on the database.

Project URL = "https://xybccoipttcvmdniwysj.supabase.co"

Public Key =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Inh5YmNjb2lw dHRjdm1kbml3eXNqIiwicm9sZSI6ImFub24iLCJpYXQiOiJlYXN0IiwiaWF0Ij0iMTY5ODc2NH0.qft8IvKBxpEz W7Uh1D4uDdGafhHzbh7fWlfi17B5nKA"

Schema Overview

Below is a list of all tables in the database, with a short description of what their purpose is and what data they store. For explanations on Primary/Foreign keys & Data Types, refer to the [ERD Diagram](#) section of the software manual.

Table Name	Description
Users	Stores all users who have created an account via Clerk. Each user can be identified by their unique userID.
UserInstitutions	Stores the link between users and any institution they belong to.

Table Name	Description
UserInstitutionProject	Stores the link between a user who has created a project and the institution they have created it for.
SubValues	Stores the values for a specified sub entry.
SubEntries	Stores an entry made by a user for a given category which features sub entries.
Roles	Stores the roles that users can belong to within a project.
Project	Stores a project, which can be identified by its unique projectID.
Invites	Stores all invitations for adding users to projects including: pending, accepted and rejected statuses.
Institutions	Stores all the institutions that users/projects can belong to.
FieldValues	Stores the data for an entry into a field.
CategoryFields	Stores all fields that belong to each category.
CategoryEntry	Stores an entry made by a user for a given category.
Categories	Stores all the categories that exist within a template.

Project Data Entry

When users make data entries into a project, it is important that the process below is followed so it is entered correctly in the appropriate tables.

Steps for Data Entry

1. Insert into **CategoryEntry**

Insert a new row into the **CategoryEntry** table. You will need to specify the following:

- **projectID**: The ID of the project.
- **categoryID**: The ID of the category for which the entry is being created.

This creates a new data entry for a specific category within the project.

2. Insert into **FieldValues**

Insert a new row into the **FieldValues** table. You will need to specify the following:

- **CategoryEntry.EntryID**: Foreign key linking the data entry to the **CategoryEntry**.
- **CategoryFields.fieldID**: Foreign key linking to the specific field within the category.

This links the data entry to the category and field it belongs to.

3. Repeat for all fields in the category

For each field within the category, repeat **Step 2** to ensure all field values are stored.

To view the code which carries out these queries to insert data in the relevant tables for an entry, [see line 577 in **database.tsx**](#)

Database Queries

The table below lists key database queries that can be found in [database.tsx](#), as well as their purpose. These queries should be reused throughout the codebase to improve code reability for all developers.

Query	Description
checkAndAddCategories	Checks if the given categories exist in the database. If not, adds them and creates their associated fields
createCategoryFields	Creates fields for a list of categories by inserting fields into the CategoryFields table
getProjectName	Fetches the project name from the Project table using the stored projectID in localStorage
createProjectQuery	Creates a new project in the Project table and links it to the user and institution
createUserInstitutionProjectQuery	Links a user to a project under a specific institution by inserting an entry into the UserInstitutionProject table
getInstitutionID	Retrieves the ID of an institution based on its name from the Institutions table
getUsersInstitutions	Fetches a list of institutions associated with the logged-in user
inviteUserToProject	Sends an invitation to a user by inserting an entry into the Invites table
getSentInvites	Retrieves all invitations sent by the given user
getPendingInvites	Fetches all pending invitations for a user
acceptInvite	Accepts an invitation, adds the user to the project in UserInstitutionProject, and updates the invite status to "accepted"
rejectInvite	Rejects an invitation by updating the invite status to "rejected"
getUsersProjects	Fetches all projects associated with the logged-in user
removeCollaborator	Removes a user from a project by deleting their entry from the UserInstitutionProject table
createCategoryEntry	Creates a new category entry in the CategoryEntry table. If the category doesn't exist, it inserts into the Categories table first
insertFieldValues	Inserts values into the FieldValues table for a given entryID and categoryID. If a field doesn't exist, it creates it first
createEntry	Uses 'createCategoryEntry' and 'insertFieldValues' to insert forms into the database
createEntryWithSubEntry	Creates an entry that contains a sub-entry and adds corresponding field values

Query	Description
addWithSubEntry	Handles inserting field values for an entry and its sub-entries, linking them together
createSubEntry	Inserts a new sub-entry record and returns its ID
addToSubValues	Associates sub-entry IDs with value IDs in the SubValues table
getSubEntriesForEntry	Retrieves all sub-entries related to a given entry and their associated field values
getFieldNameAndValue	Fetches the field name and value for a given value ID
getFieldName	Retrieves the field name corresponding to a given field ID
getRoles	Fetches all available roles from the database
getCategoryID	Retrieves the category ID using category name and project ID
getAllCategoryEntries	Fetches all entry IDs for a given category
haveSubEntry	Checks whether an entry has sub-entries
getAllSubEntries	Retrieves all sub-entry IDs for a given entry
getValueIDsOfSubEntry	Fetches value IDs associated with a sub-entry
getSubValue	Retrieves a value from a sub-entry based on field ID
getFieldID	Retrieves a field ID using a category ID and field name
getValue	Fetches a value for a given entry ID and field ID
getValueID	Retrieves the value ID for a given entry ID and field ID, creating a blank value if none exists
createBlankValue	Inserts a blank field value of "0" if no value exists for an entry and field ID
updateIndividualField	Updates a specific field value in FieldValues
getGlobalEntryID	Checks if a category has an entry, creating one if it doesn't exist
createBlankEntry	Creates a new blank entry for a category in a project
addInstitution	Adds an institution to Institutions and returns its ID
checkExisitingInstitution	Checks if an institution exists in Institutions and returns its ID if found
getInstitutions	Retrieves a list of institutions matching an input value
addUserInstitution	Links a user to an institution in UserInstitutions
getUserRoleInProject	Fetches the role ID of a user for a given project

Clone Database

If you need to setup your own instance of the database on Supabase, you can copy the PostgreSQL table schema from [here](#). This can then be ran as a PostgreSQL query in Supabase to generate the database used throughout the project.

[↑ Back to top](#)

Templates

Table of Contents

- [Templates](#)
 - [JSON Templates](#)
 - [Introduction to Templates](#)
 - [General Layout](#)
 - [Categories](#)
 - [Fields](#)
 - [Calculations](#)
 - [Sub Entries](#)
 - [Template Parsing](#)
 - [Template Data Types](#)
 - [How to use the Template Parser](#)
 - [Provided Functions to Operate on the Types](#)
 - [Expressions](#)
 - [How Expressions are Formed](#)
 - [Elements You Can Include](#)
 - [How Variables are Formed](#)
 - [Expression Parsing](#)
 - [How Variables are Evaluated](#)
 - [Expression Evaluation Steps](#)
 - [Prepping the Expression](#)
 - [Conversion to RPN](#)
 - [Basic Evaluator](#)
 - [Updating the Database](#)
 - [Different Evaluator Types](#)
 - [Local](#)
 - [Global](#)
 - [SubGlobal](#)
 - [How to use Evaluators in General](#)
 - [How to Add More Operations if Required](#)

JSON Templates

Introduction to templates

The data is stored inside the database as defined by the template that is assigned to the project. Templates themselves are not meant to be user-facing, they should just be given a name which denotes which project

funding type they represent. It is up to the PI of the project to know what the contents of the project would look like. Therefore it is up to the developers to provide the templates that a PI may want to create projects for.

General Layout

Templates are stored in the form of a JSON file within the database, and when a project is created by a PI, this template is copied into the project's template entry inside the database (pending). The result of this is that all projects have their own copy of their template, such that if the master copy changes, their database layout remains intact. The top level of the JSON object has one object and one attribute as shown below:

```
{
  "templateName" : "Sample Name",
  "Categories" :
  {
    ...
    ...
  }
}
```

The template name is simply the display name of the template and should be recognised by a potential PI of a project. This will allow them to choose the appropriate template type for their project. The categories section (explained below) defines how the data should be structured and what tables/fields the user should be able to interact with.

Categories

In the template the "Categories" section is made up of a list of objects, each forming single category. A category defines a table inside the database where the user would like to store data. These categories are separate from each other and cannot be linked together to form a joined table. However data can be linked together by way of calculations (as described in the Calculations section below). In the JSON templates, the categories section will be populated with objects as below:

```
"Categories" :
{
  "Category Abbreviation" :
  {
    "Name" : "Underscore_Separated_Name",
    "HasSubEntry" : Boolean Value,
    "Fields" :
    {
      ...
    },
    "SubEntries" :
    {
      ... (Only present if "HasSubEntry" is True)
    },
  },
}
```

```

        "Calculations" :
        {
            ...
        }
    },
    ...
}

```

The "Category Abbreviation" can be anything, it doesn't impact the internal workings of the template, nor does it get displayed to the user and so it can be disregarded. Internally, a category has two attributes and between 2 and 3 objects. The number of objects is dependent on one of the attributes. The first attribute is the "Name", which stores the name of the attribute and must not use spaces. Instead you should use underscores (which can be easily removed for display). Underscores are used to allow expressions (discussed later) to reference category names easily. The second attribute is "HasSubEntry", which is used to express whether the category contains "SubEntries". (SubEntries are discussed in more detail later on). If this attribute reads true, then the category contains a "SubEntries" object, else this object can be omitted. The two remaining objects are "Fields" and "Calculations". Fields holds the actual data points that can be added as part of a single entry to the database. Calculations holds the predefined ways in which values can be added together or processed within that category. Both of these are discussed in further detail later on.

Fields

In the template the "Fields" section of a category is again made up of a list of objects, each forming a single field in the database as part of the table defined in the category. In the JSON template, the fields section will be populated with objects as below:

```

"Fields" :
{
    "Field Abbreviation" :
    {
        "Name" : "Underscore_Separated_Name",
        "Prefix" : "Symbol or word to prefix the header",
        "Value" : "Placeholder value in entry",
        "Postfix" : "Symbol or word to postfix header",
        "Type" : "Data type of the field",
        "DisplayVisibility" : "If visible on data retrieval as boolean",
        "EntryVisibility" : "If visible on data entry as boolean"
    },
    ...
}

```

The "Field Abbreviation" can be anything, it doesn't impact the internal workings of the template, nor does it get displayed to the user and so it can be disregarded. Internally a field has 7 attributes and no internal objects. The first attribute is "Name", which must not contain spaces, underscores should be used instead (which can be removed for display). Underscores make it easier for the expression parser (discussed later) to recognise variables which contain the field names. The next two are "Prefix" and "Postfix" which will append the supplied characters to the front and end of the header respectively. For example, placing £ in the prefix

means you can tell users that the field's data should be in pounds. The "Value" attribute stores what will be placed in the textbox for entry as a placeholder, giving the user extra instruction about what to fill that field in with. Next, the "Type" attribute is used to define the data type of the field such as "String", "Int" etc. The final two are "DisplayVisibility" and "EntryVisibility", which define what data should be visible and what data shouldn't be visible. Display defines whether the data should be displayed as read in from the database, or if that column should be hidden. This will mostly be useful to hide some intermediate calculation results (as explained later). Entry defines whether the user should be given a textbox to enter the data into the database in the first place. This is useful when you don't want a user to enter data into the result of a calculation as the data will get overwritten. Both of these attributes are booleans.

Calculations

In the template the "Calculations" section of a category is again made up of a list of objects, each forming a single calculation that can be carried out in that category. In the JSON template, the calculations section will be populated with objects as below:

```
"Calculations" :
{
  "Calculation Abbreviation" :
  {
    "Name" : "Underscore_Separated_Name",
    "Expression" : "String expression",
    "Type" : "How it should be evaluated",
    "Output" : "Name of field to output to"
  },
  ...
}
```

The "Calculation Abbreviation" can be anything, it doesn't impact the internal workings of the template, nor does it get displayed to the user and so it can be disregarded. Internally a calculation has 4 attributes and no internal objects. The first attribute is "Name". This should not contain spaces and should instead use underscores to maintain consistency with the field names. Next is the "Expression" which stores what calculation should be carried out. Further details on how these work are available in the Expression section. Then there is the "Type", which defines how the calculation should be carried out. This can either be "Local", "Global" or "SubGlobal" and how these work will be explained in the Expression Parsing section. Finally there is the "Output" which states which field in the category the output should be saved to. This must be a field within the category.

Sub Entries

A sub entry is a small collection of fields inside a category for which a single database entry may require them to have more than one value, and the groups of values entered together should be maintained. For example, in a "Travel_Costs" entry, you may require more than one entry for "What" and "Amount" for a single main entry of "Days" etc. Sub entries provide a way to allow for multiple "What" and "Amount" pairs to be added along with a single entry for the category. Not every category will have a "SubEntries" section. Any category that needs a "SubEntries" section must also have the "HasSubEntry" flag set to true, else the sub entries will be ignored. The "SubEntries" section is made up of a list of objects in exactly the same way that "Fields" is, and

also contains a list of fields which follow the same format as in the "Fields" section. For data entry, retrieval and expressions to function as intended, sub entries and fields should not be given the same name within the same category

Template Parsing

Template Data Types

Provided to make interacting with the template's information easier are a number of types (located in types.ts) which capture the sections of the template. Generating these types is explained in the next section.

We will look at the types in a bottom up fashion instead of the top down fashion they are generated in, so that those which are expressed in terms of another type appear later than the type it contains.

This stores the fields of a category within the template, holding each of its attributes. This is also the type used to hold a sub entry as a sub entry contains all of the same attributes as a field.

```
export interface FieldType {
  name: string;
  prefix: string;
  value: string;
  postfix: string;
  type: string;
  displayvisible: boolean;
  entryvisible: boolean;
}
```

This stores the calculations of a category within the template, holding each of its attributes.

```
export interface CalculationType {
  name: string;
  expression: string;
  type: string;
  output: string;
}
```

This stores the categories of a template, holding each of its attributes. It contains lists of FieldType and CalculationType for the fields, sub entries and calculations of the category. If hassubentry is False, then the subentries will be []

```
export interface CategoryType {
  name: string;
  hassubentry: boolean;
  fields: FieldType[];
  calculations: CalculationType[];
  subentries: FieldType[];
}
```

This stores the overall template, holding the name attribute and the list of the CategoryType's that make up the categories of the template.

```
export interface TemplateData {
  templateName: string;
  categories: CategoryType[];
}
```

How to use the Template Parser

In order to parse the JSON templates into a consistent format for the code to oeprate on, there is a template parser in newTemplateParser.tsx. The main function of this is to turn the JSON template into a data structure formed of bespoke types. The types themselves are descibed in more detail in the previous section. The parser also provides some functions for operations on these structures for some common tasks such as listing the names of the categories in said template.

The most important function of the template parser is:

```
/**
 * Generates a TemplateData structure from a raw JSON template file, gathering the
 * CategoryTypes and other internal structures
 * @param template The raw template JSON that has been read in from readJsonFile
 * @returns A complete TemplateData structure which contains the categories list
 *         filled out
 */
export function getTemplate(template: any): TemplateData {
  let temp = {
    templateName: template.templateName,
    categories: getCategoriesFromRaw(template),
  };

  return temp;
}
```

which generates the main TemplateData object in a top down fashion, storing all of the provided information in the JSON in a simpler format. It operates on the template's text either read in from a file, or more likely from the database itself. This starts a chain of function calls which find each category inside the JSON and generate the lists of its fields etc, and then creates the list of categories for the TemplateData type. Elsewhere in the code these should never be used as you should already have use the "getTemplate" function which handles the raw template for you.

Provided functions to operate on the types

Function Name	Parameters	Return	Description
---------------	------------	--------	-------------

Function Name	Parameters	Return	Description
getCatNames	TemplateData	string[]	Returns a list of names of the categories inside the given template
getFieldsNames	CategoryType	string[]	Returns a list of names of the fields inside the given category
findCatObject	string, CategoryType[]	CategoryType	Returns the category with the given name from the list of CategoryType given using a linear search
findFieldObject	string, CategoryType	FieldType	Returns the field with the given name from the list of FieldType inside the given category using a linear search
findCalcObject	string, CategoryType	CalculationType	Returns the calculation with the given name from the list of CalculationType inside the given category using a linear search
getSubEntryNames	CategoryType	string[]	Returns a list of names of the sub entries inside the given category, or the empty list if no sub entries are present
findSubEntryObject	string, CategoryType	FieldType	Returns the sub entry with the given name from the list of FieldType (sub entries) inside the given category using a linear search. Returns an empty FieldType if there are no subentries

Expressions

How expressions are formed

Expressions are stored as strings as defined in the template. You should simply write out the expression that must be calculated, including brackets where necessary (following "order of operations" rules). You do not have to worry about where spaces occur within the expression as long as they are not inside of a variable. You also don't have to worry about the output and = signs as this is handled inside the "Output" attribute of the calculation as explained in the calculations section of the template manual. An example expression is given inside the "How variables are formed" section below.

Should you wish to include calculations which sum up parts of entries together, this is not defined inside an expression, you should instead look at "Different Evaluator Types" inside the expression parsing section.

How expressions handle dates needs to be added, but implementation is not yet complete and method not fully decided

Elements you can include

Elements	Symbol
----------	--------

Elements	Symbol
Add	+
Subtract	-
Multiply	*
Divide	/
Exponential	^
Brackets	()
Variables	Category:Field

To add more operations, see the "How to add more operations if required" section later

How variables are formed

A variable is simply formed of the category you are currently in and the field within that category, separated by a ;, for example

```
"Expression" : "Travel_Costs:Days*Travel_Costs:Accomodation +
Travel_Costs:Days*Travel_Costs:Sustinance"
```

where the variables are "Travel_Costs:Days", "Travel_Costs:Accomodation" and "Travel_Costs:Sustinance".

The category in the variable does not have to refer to the current category you are writing an expression for, but as variables are found on an entry-level, it is best to use a "Global" calculation if you refer to outside the current category. This will be further explained in the "Expression Parsing" section.

Expression Parsing

The code which evaluates the expressions is given inside "expressionParser.tsx". It operates by calling the "categoryCalculation" function whenever a categories calculations need to be updated. It will run thorough each of the calculations and carry out the calulations within each, choosing which type of evaluator to use and what entries need to be included in the calculation.

How variables are evaluated

A single variable is evaluated by first breaking down the variable, and then finding the variable within the database. This is done with the "parseVariable" and "getVariable" functions. First you call parseVariable giving the variable name, the ID of the project, the ID of the entry and whether its a sub entry or not. If it is a sub entry, then the entryID should be used as a sub entry ID instead. (This is explained further in the SubGlobal evaluator type later on). Then the string is split to extract the category and field name from around the :. This then calls getVariable with the two extracted names along with the rest of the parameters. getVariable will then find the category and field ID inside the database and will call on of two functions, either getValue or getSubValue (depending on the status of the isSubEntry parameter), giving both the entry and field IDs. This will return the value of the variable in the provided context for use in the calculation.

Expression evaluation steps

Prepping the expression

The first step in the process of evaluating an expression is to prep the expression. This is the process of turning the expression into an array of tokens in the expression, where each token is a string that is either a number, variable or operator that has been found in the expression. This is done by the function "prepString" which takes an string expression and returns a list of strings. prepString works by looping through each character in the expression.

Character type	How it is parsed
isSpace	Skip the character and move on
isOperator	Push the token to the output list and move on
isNumeric	Continue to loop through, gathering all numbers into an extra list, then when you reach a non-numeric character, place the whole number into the output list and move on, going back to the character you just parsed that wasn't a number
isAlphabet	As above, but with isAlphabet characters instead of numbers

These are the function which define what type each parsed character belongs to. Notably isAlphabet only includes a-z, A-Z, : and _ all other characters are ignored.

```
function isSpace(element: string): boolean {
  // Checks if an elements is a space or a blank string
  if (element == " " || element == "") {
    return true;
  }

  return false;
}

function isOperator(value: string): boolean {
  let ret: boolean = false;

  Object.values(operators).forEach((operator) => {
    if (operator.symbol === value) {
      ret = true;
      return;
    }
  });

  return ret;
}

function isNumeric(c: string): boolean {
  if ((Number.isFinite(+c) || c === ".") && !isSpace(c)) {
    return true;
  }
}
```

```
    return false;
}

function isAlphabet(c: string): boolean {
  if (
    ((c >= "a" && c <= "z") ||
     (c >= "A" && c <= "Z") ||
     c === ":" ||
     c === "_") &&
    !isSpace(c)
  ) {
    return true;
  }

  return false;
}
```

Once the full expression has been parsed, you will be left with a list of the tokens in the expression. Using the example in "How variables are formed" you would get:

```
[
  "Travel_Costs:Days",
  "*",
  "Travel_Costs:Accomodation",
  "+",
  "Travel_Costs:Days",
  "*",
  "Travel_Costs:Sustinance",
];
```

note the spaces have been removed.

Conversion to RPN

Before the expression can be properly evaluated, is must be converted in Reverse Polish Notation (also known as postfix expressions). This is carried out using the "expressionToRPN" function which takes the whole string expression and returns the expression as a list of strings in postfix form. Note that this function takes the expression string and not the prepped list of tokens as it carries out the prepString itself. It converts an expressino to RPN using the Shunting Yard algorithm utilising an operator stack and an output queue. It will loop through the prepped tokens and will do one of four things:

Token type	Operation
"("	Pushes to operator stack
)"	Pops from operator stack to output until a "(" is found

Token type	Operation
isOperator	Pops from operator stack to output until it finds an operator with less precedence then pushes to operator stack
Otherwise	Must be a variable or number so push to output

Basic evaluator

Now the expression can be evaluated. This is done by the "basicEvaluator" function which takes the postfix expression, the projectID, entryID and isSubEntry and returns the number asynchronously using a stack for the numbers. You would never typically use this function alone as it would be a part of an evaluator which are described alter on. It requires the IDs of the project involved as it must parse the variables within the expression and gather the values from the database. To evaluate an expression it will loop through the postfix expression:

Token type	Operation
isAlphabet	Call parse variable to get the numerical value and push to the number stack
isOperator	Pop two values from the stack, convert them into floating point numbers then apply the operation in the token
Otherwise	It is a number and should be placed on the number stack

Once the whole expression has been dealt with, there should be a single number remaining on the stack which is returned else 0 is returned.

Updating the database

Once the expression has been evaluated the value should be placed into the database using the "addResultToDataBase" function which takes the entryID, the output field as a string, the categoryID and the number that is must be updated with. It is assumed that the results to a calcaultion cannot be inside a sub entry and so this should be avoided. The function will simply find the field and entry IDs then update the field inside the database using the "updateIndividualField" query

Different evaluator types

There are three different types of evaluator provided which are described in the section below. There are different types of evaluator to allow you to calculate within an entry, such as person months, to sum up values such as totals and to sum up sub entries such as in "Travel_Costs". These all require the basicEvaluator to be applied in different ways, possibly with loops etc so are split into different evaluators. Which evaluator is applied to which calculation is defined in the type of the calculation within the template and must contain either "Local", "Global", or "SubGlobal". While different they all follow a similar pattern. Firstly the expression in the given CalculationType must be converted to postfix using "expressionToRPN", then you must call the "basicEvaluator" either in a loop of some form, or just on its own, dependent on the type of evaluator, then finally you must return the result of the calculation.

Local

A "Local" calculation type is one that happens within a single database entry (think within the fields of a single row). Its evaluator therefore is very simple, just converting to RPN then calling the basicEvaluator and returning the result. To run a local evaluation for a whole category, you must place it inside a loop which calls the evaluator once for each entryID in the chosen category as per "categoryCalculations" function, adding to the database after each one. These can appear in any category, even among global calculations should they be required.

Global

A "Global" calculation type is one that happens along all entries inside a project, used to sum up values possibly from local calculations. A calculation of this type will typically only contain a single variable, and the category it uses the entries of is determined by the first category it comes across in the string expression. It is therefore best practise to use a local calculation for small calculations then sum up the results of those rather than doing all in one go. The evaluator will get all entries of the category it determines, will convert the expression to RPN then run the evaluator for each entryID keeping a running total before returning that total. To run a global evaluation for a whole category, you can simply call the global evaluator and add to the database as per "categoryCalculations". However as global calculations are not coupled to an entry, you must call getGlobalEntryID which will maintain a singleton entry for that category to place the results of global calculations. Therefore it is advised that global calculations end up inside a category of their own such as "Totals" away from typical data entry categories.

SubGlobal

A "SubGlobal" calculation type is one that happens within a single database entry but which also contains sub entries and is used to sum up values inside the sub entry. It must therefore act as a global evaluator inside a local area. Its evaluator acts very much like the global evaluator, instead looping through sub entries for the chosen entry id, and giving the basic evaluator a flag of true instead of false. It can also include some local calculations such as adding number together as long as they appear inside the same sub entry. To run a subglobal evaluation for a whole category, you must place it inside a loop which calls the evaluator once for every entryID in the category as per "categoryCalculations". As they are still tied to an entry, they can simply add the result into the database at the entry it operated on. These can be used in any category which has a sub entry.

How to use evaluators in general

Whichever evaluator type you use, there is a general pattern to use. This involves calling the evaluator you wish to use, then adding it to the database. Depending on the evaluator you have chosen this may be carried out inside of a loop or be done inside of a single entry.

How to add more operations if required

You may wish to add more operations to expressions for more complex calculations such as a modulus operator. You can do this by updating the operators list inside "expressionParser.tsx", ensuring that the precedence and associativity is correct relative to the other operators.


```
const operators: Operator[] = [
  { symbol: "+", precedence: 1, associativity: Associativity.Left },
  { symbol: "-", precedence: 1, associativity: Associativity.Left },
  { symbol: "*", precedence: 2, associativity: Associativity.Left },
  { symbol: "/", precedence: 2, associativity: Associativity.Left },
  { symbol: "^", precedence: 3, associativity: Associativity.Right },
];
```

You must also update the switch case basic evaluator which handles applying the operators to include a case for the new operator as shown below

```
switch (token) {
  case "+": {
    val = op2 + op1;
    break;
  }

  case "-": {
    val = op2 - op1;
    break;
  }

  case "*": {
    val = op2 * op1;
    break;
  }

  case "/": {
    if (op2 == 0) {
      val = 0;
    } else {
      val = op2 / op1;
    }

    break;
  }

  case "^": {
    val = op2 ** op1;
    break;
  }
}
```

[↑ Back to top](#)